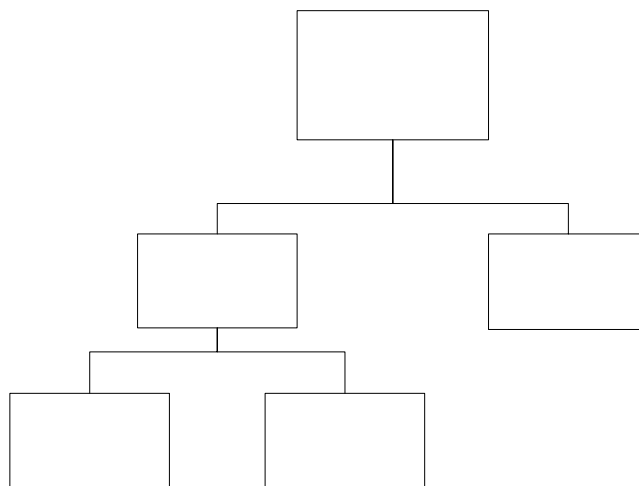


บทที่ 5

การออกแบบและการสร้างเว็บไซต์

5.1 การออกแบบ

เราสามารถออกแบบเว็บไซต์ได้ดังไดอะแกรมในรูปที่ 5-1



รูปที่ 5-1 ไดอะแกรมของเว็บไซต์

เว็บแผนที่นี้จะแบ่งออกเป็น 2 ส่วนใหญ่ๆ คือ

1. ส่วนติดต่อกับผู้ใช้ (User Interface) จะใช้โปรเว็บในการสร้างเว็บโดยจะแบ่งออกเป็น 2 ส่วนคือส่วน
วินิจัยจะใช้โปรล็อกและฐานข้อมูลจะใช้โปรดาต้า
2. ส่วนแสดงผลภาพแผนที่ จะใช้ Servlet ในการนำภาพแผนที่ที่ได้มาแสดงผลบนเว็บ

5.2 การสร้างเว็บไซต์

การสร้างเว็บเราได้แยกออกเป็นส่วนได้ดังนี้

5.2.1 โปรเว็บ

จะทำหน้าที่รับข้อมูลเข้ามาเพื่อส่งไปยังส่วนวินิจัยแล้วนำผลที่ได้นำมาแสดงผลในรูปของเว็บเพจ การแสดงผลรวมไปถึงภาพแผนที่ที่ได้จากผลการวินิจัยและการสร้างจาวาสคริปต์ด้วย แบ่งการแสดงผลออกตามการทำงานของเว็บไซต์ได้ดังนี้

1. การค้นหาสถานที่ จะเป็นแสดงผลรายชื่อสถานที่และประเภทที่ต้องการค้นหา ใคร่เอนท์สามารถเลือกได้เพียงสถานที่เพียงแห่งเดียวเท่านั้นในการแสดงผลโดยใช้แท็ก INPUT TYPE= RADIO เช่น
<input type=radio name="road_id" value="road_lad">ลาดพร้าว

2. การค้นหาสถานที่ใกล้เคียง เมื่อผู้ใช้ข้อมูลประเภทสถานที่และรัศมีการค้นหาแล้ว โปรแกรมก็จะแสดงผลเพื่อให้ผู้ใช้ป้อนข้อมูลของจุดอ้างอิงหรือเลือกที่ตัวแผนที่ได้เลยก็สามารถระบุจุดอ้างอิงได้ด้วย หลังจากนั้นเมื่อได้รับข้อมูลทั้งหมดก็นำไปประมวลผลในส่วนวินิจัยแล้วแสดงผลรายชื่อสถานที่ค้นหาได้และแสดงตำแหน่งของจุดอ้างอิงและสถานที่ต่างๆในแผนที่ด้วย
3. การค้นหาถนนสายหลัก จะหลักการทำงานคล้ายกับการค้นหาสถานที่แต่จะต่างกันที่ส่วนแสดงผลลัพธ์โดยจะแสดงเส้นทางของถนนนั้นๆและชื่อถนนด้วย
4. การค้นหาเส้นทางที่สั้นที่สุด จะแสดงผลเพื่อรับตำแหน่งของจุดเริ่มต้นและจุดหมาย และแสดงผลการวินิจัยออกเป็นเส้นทางที่ได้ในแผนที่และแสดงรายละเอียดของเส้นทางด้วย

5.2.2 ส่วนวินิจัย

แบ่งออกตามหน้าที่การทำงานของเว็บไซต์

1. การค้นหาสถานที่ จะเป็นการค้นหาสถานที่และประเภทที่กำหนดโดยจะเก็บเป็นลิสต์ ส่วนการนำไปใช้งานอาจจะมีการคัดแปลงเพื่อความเหมาะสมกับงาน

`search_PlaceList( PlaceType, Name , List ):-`

```
findall( PlaceID ,
(
place( PlaceID, _, _, Type, PlaceName),
sub_string( Name, PlaceName ) ,
),
List).
```

2. การค้นหาสถานที่ใกล้เคียง จะเป็นการค้นหาสถานที่ที่เป็นประเภทที่กำหนดและอยู่ในระยะที่กำหนด โดยจะเก็บเป็นลิสต์

`nearest_place( X1, Y1, Range, PlaceType, PlaceList ):-`

```
findall( [PlaceID,PlaceName],
(
place( PlaceID, X2, Y2, PlaceType, PlaceName ),
dist_PointToPoint( X1,Y1, X2, Y2, Dist ),
Dist < Range
),
PlaceList ).
```

3. การค้นหาถนนสายหลัก จะหลักการทำงานคล้ายกับการค้นหาสถานที่

`search_RoadList( Sub, List ):-`

```
findall( RoadID ,
(
```

```

road( RoadID, RoadName, _, _, _ ),
sub_string( Sub, RoadName
),
List ).

```

4. การค้นหาเส้นทางที่สั้นที่สุด จะเป็นการค้นหาเส้นทางที่สั้นที่สุดโดยวิธีเอ-สตาร์ซึ่งมีรายละเอียดอยู่ในหัวข้อ 4.8 โดยนำหลักการดังกล่าวมาเขียนในภาษาโปรล็อกดังนี้

```
searchNode( PointID, PointID, [], 0 ).
```

```
searchNode( StartPointID, GoalPointID, Solution, Distance ):-
```

```

    astar_node( [ [ 0, 0, StartPointID ] ], Solution, GoalPointID, Distance ),
    !.

```

```
astar_node( [ [ _, Distance, Goal|Path ] | _ ], [ Goal|Path ], Goal, Distance ).
```

```
astar_node( [ Path|Paths ], Solution, GoalNode, Distance ) :-
```

```

    extendNode( Path, NewPaths, GoalNode ),
    insertAll( NewPaths, Paths, Paths2 ),
    astar_node( Paths2, Solution, GoalNode, Distance ).

```

```
extendNode( [ F, G, ThisPointID|Path ], NewPaths, GoalPointID ) :-
```

```

    setof( [Fn, Gn, NewPointID, ThisPointID|Path],
        (
            linked( ThisPointID, NewPointID ),
            \+ member( NewPointID, [ ThisPointID|Path ] ),
            dist_PtIDToPtID( ThisPointID, NewPointID, Dist ),
            Gn is G+Dist,
            heuristicFn( NewPointID, GoalPointID, H ), % Heuristic Function
            Fn is Gn+H
        ),
        NewPaths ).

```

```
linked( A, B ):-
```

```

    ( path( _, A, B ); path( _, B, A ) ).

```

```
dist_PtIDToPtID( PtId1, PtId2, Dist ):-
```

```

    position( PtId1, X1, Y1 ),
    position( PtId2, X2, Y2 ),
    dist_PointToPoint( X1,Y1, X2, Y2, Dist ).

```

```
heuristicFn( PtId1, PtId2, Dist ):-
```

```

    dist_PtIDToPtID( PtId1, PtId2, Dist ).

```

insertAll([], Paths, Paths).

insertAll([Path|Paths], Paths2, Paths3) :-

insertOne(Path, Paths2 , Paths4),

insertAll(Paths,Paths4,Paths3).

insertOne(Path, [], [Path]).

insertOne([F, G|Path], [[F2, G2|Path2]|Paths], [[F, G|Path], [F2, G2|Path2]|Paths]):-

$F < F2$.

insertOne([F, G|Path], [[F2, G2|Path2]|Paths], [[F2, G2|Path2]|Paths2]):-

insertOne([F, G|Path], Paths, Paths2).

5.2.3 ฐานข้อมูล

จะเก็บข้อมูลเฉพาะฐานข้อมูลเท่านั้นหมายถึง fact เท่านั้นที่สามารถเก็บไว้ในฐานข้อมูลได้จะมีด้วยกันทั้งหมด 8 ตารางดังนี้

1. ตาราง POSITION เป็นตารางที่เก็บตำแหน่งของจุด(Node)ต่างๆ

ชื่อคอลัมน์	รายละเอียด
POINTID	รหัสจุด
X	ลองติจูด
Y	ละติจูด

ตารางที่ 5-1 ตาราง POSITION

2. ตาราง PATH เป็นตารางที่เก็บชื่อพาทและของจุด 2 จุดต่างๆ

ชื่อคอลัมน์	รายละเอียด
PATHID	รหัสพาท
POINTID1	รหัสจุด
POINTID2	รหัสจุด

ตารางที่ 5-2 ตาราง PATH

3. ตาราง PLACE เป็นตารางที่เก็บรายละเอียดสถานที่

ชื่อคอลัมน์	รายละเอียด
PLACEID	รหัสสถานที่
X	ลองติจูดที่แสดงชื่อถนน
Y	ละติจูดที่แสดงชื่อถนน
PLACETYPE	ประเภทของสถานที่
PLACENAME	ชื่อสถานที่

ตารางที่ 5-3 ตาราง PLACE

4. ตาราง PLACETYPE เป็นตารางที่เก็บประเภทสถานที่

ชื่อคอลัมน์	รายละเอียด
PLACETYPE	รหัสประเภทสถานที่
PLACETYPENAME	ประเภทของสถานที่

ตารางที่ 5-4 ตาราง PLACETYPE

5. ตาราง ROAD เป็นตารางที่เก็บรายละเอียดถนน

ชื่อคอลัมน์	รายละเอียด
ROADID	รหัสถนน
ROADNAME	ชื่อถนน
ROADTYPE	รหัสประเภทถนน
X	ลองติจูด
Y	ละติจูด

ตารางที่ 5-5 ตาราง ROAD

6. ตาราง ROADPATH เป็นตารางที่เก็บบอกได้ว่าถนนมีจุดอะไรบ้าง

ชื่อคอลัมน์	รายละเอียด
ROADID	รหัสถนน
POINTID	รหัสจุด
TYPE	ประเภทของจุด

ตารางที่ 5-6 ตาราง ROADPATH

7. ตาราง ROADTYPE เป็นตารางที่เก็บชื่อพาหนะและของจุด 2 จุดต่างๆ

ชื่อคอลัมน์	รายละเอียด
ROADTYPE	รหัสประเภทถนน
ROADTYPENAME	ชื่อประเภทถนน

ตารางที่ 5-7 ตาราง ROADTYPE

8. ตาราง CROSSROAD เป็นตารางที่เก็บรายละเอียดทางแยกต่างๆ

ชื่อคอลัมน์	รายละเอียด
CROSSID	รหัสทางแยก
POINTID	รหัสจุด

ตารางที่ 5-8 ตาราง CROSSROAD

5.2.4 Servlet

5.2.4.1 เหตุผลการใช้งาน Servlet

เนื่องจาก WIN-PROLOG สามารถสร้างไฟล์ภาพเป็นแบบเวกเตอร์แต่ไม่สามารถแสดงผลในเว็บเบราว์เซอร์ได้ จึงต้องมีโปรแกรมหนึ่งที่เป็นตัวกลางในการแปลงไฟล์ดังกล่าวให้เป็นไฟล์ประเภทอื่นเช่น GIF, JPEG เป็นต้น แต่เราเลือกที่จะแปลงเป็นไฟล์ GIF เนื่องจากหาโค้ดแปลงเป็นไฟล์ GIF ที่เป็น Open source ได้ โปรแกรมที่เรานำมาเป็นตัวกลางในการแปลงเราเลือกใช้ JavaServlet เพราะสามารถแปลงแล้วส่งเป็นข้อมูลภาพโดยที่ไม่ต้องเก็บเป็นไฟล์ก่อนหรือเรียกว่าแปลงแบบไดนามิก

5.2.4.2 การทำงาน

ลักษณะการทำงานของ Servlet เป็นดังไดอะแกรมในรูปที่ 5-2

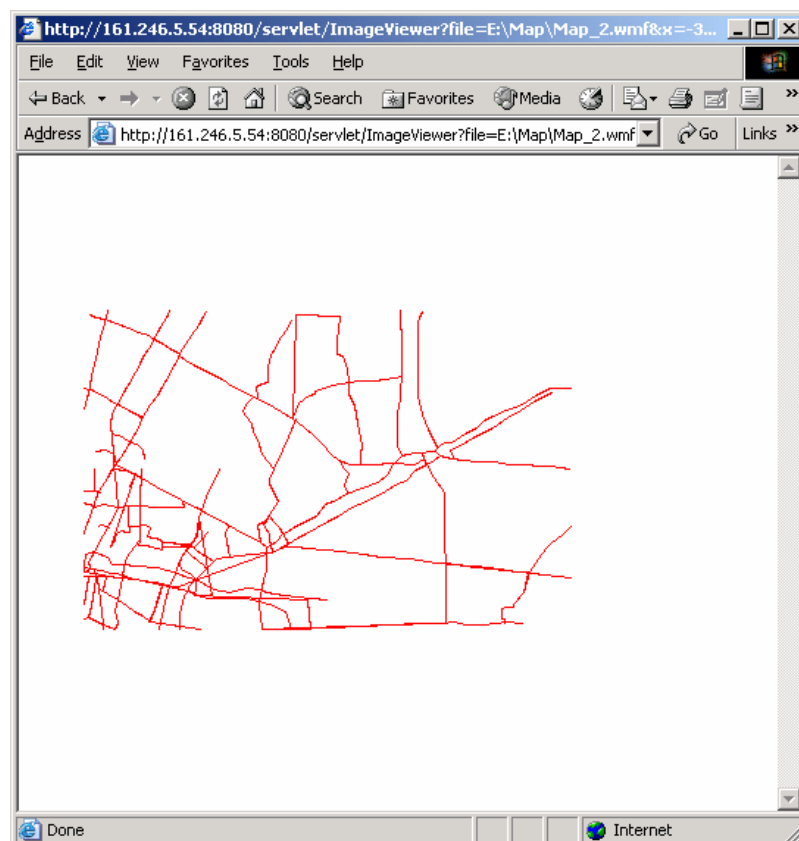


รูปที่ 5-2 ไดอะแกรมการทำงานของ Servlet

เนื่องจากไฟล์ WMF เป็นไฟล์แบบเวกเตอร์ดังนั้นเมื่อมีการแปลงไฟล์ Servlet จะทำการอ่านไฟล์ WMF เพื่อจะวาดภาพหลังจากนั้นก็แปลง ภาพที่ได้เป็นข้อมูลในรูปแบบของไฟล์ GIF ดังในรูปที่ 5-3

5.2.4.3 ปัญหาในการสร้าง

การหาข้อมูลรูปแบบของไฟล์ WMF ในการทำนั้นยากมากเนื่องจากเป็นรูปแบบที่ไม่ใครซอฟต์แวร์ค้นคิดขึ้นมาแต่ไม่มีข้อมูลของไฟล์ดังกล่าว ดังนั้นจึงต้องค้นหาตัวอย่างโปรแกรมที่สามารถแสดงผลไฟล์ WMF ได้ ซึ่งเป็นโปรแกรม WmfView [8] แล้วต้องนำโปรแกรกดังกล่าวมาดัดแปลงให้สามารถทำงานได้



รูปที่ 5-2 ตัวอย่างการแปลงไฟล์ WMF ผ่าน Servlet